PROJECT HANDOVER DOCUMENT

Vertex — Team Links Portal (v2)

April 2026 | Scipher Operations

CONFIDENTIAL — INTERNAL USE ONLY

Classification: Confidential

1. Project Overview

Vertex is an internal web portal built for Vestas / Scipher Operations that acts as a centralised team links hub. Authenticated users land on a role-gated home screen that shows only the team cards relevant to them. Clicking a card navigates to a team-specific page that renders links, tools, and external service shortcuts read live from an Excel workbook (team_links.xlsx). An admin panel provides full user, team, and role management.

Repository	vertex - Repos
Active Codebase	v2/ directory
Framework	Python / Flask 3.1.2
Database	PostgreSQL (local or remote, auto-failover)
Auth	Flask-Login + Werkzeug bcrypt
Rate Limiting	Flask-Limiter (in-memory storage)
Data Source	Excel workbook — team_links.xlsx
Python Version	3.12+ (zoneinfo used in models)
Licence	See v2/app/LICENSE
Support Email	Configured via SUPPORT_EMAIL env var

2. Repository & Directory Structure

The repository root contains only configuration files and the v2/ directory. A version 1 codebase appears to have existed but is not committed — only v2 is present.

```
Vertex/                          ← repo root
├── .gitignore
├── README.md                     (minimal — just 'Vertex')
└── v2/                          ← ACTIVE CODEBASE
    ├── run.py                   Entry point (debug mode)
    ├── wsgi.py                  WSGI entry point for Gunicorn/uWSGI
    ├── check_user.py            Dev utility: verify user credentials
    ├── team_links.xlsx          Excel data source (also in app/static/data/)
    └── app/                     Flask application package
        ├── __init__.py          App factory (create_app)
        ├── app.py               Main blueprint — all primary routes
        ├── auths.py             Auth blueprint — /admin/* routes
        ├── config.py            Config class + DB failover logic
        ├── models.py            SQLAlchemy models (User, Team, UserTeam)
        ├── helpers.py           Excel loader + link/icon/preview processor
        ├── helpers copy.py      Stale copy of helpers.py (safe to delete)
        ├── security.py          Flask-Limiter rate limit setup
        ├── errors.py            Error handlers + SQL injection detection
        ├── create_tables.py     Standalone DB bootstrap + Excel user seeder
        ├── seed_users.xlsx      Bootstrap user data (email/role/password/preset)
```


Classification: Confidential

```
        ├── sample-env.txt       .env template (rename to .env and fill in)
        ├── requirements.txt     Python dependencies
        ├── LICENSE
        ├── static/
        │   ├── css/styles.css
        │   └── data/
        │       ├── logo/        Tool & team logos (PNG, SVG, JPG)
        │       ├── app-screen/  Application preview screenshots
        │       ├── team_links.xlsx  (duplicate of v2/team_links.xlsx)
        └── templates/
            ├── base.html        Master layout (Bootstrap 5)
            ├── index.html       Role-gated home / team card selector
            ├── team.html        Team-specific link cards
            ├── login.html       Login form
            ├── admin_panel.html User + team admin table
            ├── create_user.html Create user form
            ├── edit_user.html   Edit user form
            └── errors/          400,401,403,404,405,413,429,500
```

3. Tech Stack & Dependencies

Language	Python 3.12+
Web Framework	Flask 3.1.2
ORM	Flask-SQLAlchemy + SQLAlchemy
Database Driver	psycopg2
Auth	Flask-Login + Werkzeug (bcrypt hashing)
Rate Limiting	Flask-Limiter (in-memory)
Data Processing	pandas + openpyxl (Excel ingestion)
HTTP Client	requests 2.32.5
Config	python-dotenv
Frontend	Bootstrap 5.3 (CDN) + FontAwesome 6.5 (CDN) + custom CSS

3.1 Installation

```
cd v2
python3 -m venv venv
source venv/bin/activate     # Linux/macOS
# .\venv\Scripts\activate   # Windows
pip install -r app/requirements.txt
```

4. Configuration & Environment Variables

Copy sample-env.txt to .env in the v2/ directory and populate all values. The app will not start without a valid .env file.

Classification: Confidential

```
cp v2/app/sample-env.txt v2/.env
# then edit v2/.env
```

Variable	Example / Default	Description
xl_file	app/static/data/team_links.xlsx	Path to the Excel workbook (relative to v2/ working dir)
APP_NAME	Vertex	Injected into all templates as the application title
APP_VERSION	1.0	Version string shown in the UI
MAINTENANCE_MODE	false	Set true to redirect all traffic to maintenance page
MAINTENANCE_WINDOW	ISO 8601 interval	Informational — displayed during maintenance
SUPPORT_EMAIL	support@email.com	Shown in error pages and footer
session_secret_key	(strong random string)	Flask session signing key — MUST be a long random secret
schema_name	vertex	PostgreSQL schema that owns all tables
POSTGRES_URI	postgresql://user:pass@host/db	Primary PostgreSQL connection string
POSTGRES_local_URI	postgresql://postgres:...	Fallback local DB (used if primary unreachable)
DASHBOARD_URLS	JSON array of {title,url}	Embedded iframe dashboards (e.g. Datadog WAF)
DOMAIN	google.com,yahoo.io	Domains for SecurityScorecard lookups (if SSC integrated)
SSC_API_KEY	api_key	SecurityScorecard API token (if SSC integrated)

⚠ Bug Note: config.py's resolve_database_uri() computes the correct URI into a local variable but never returns it — SQLALCHEMY_DATABASE_URI will always be None. See Section 11 (Known Issues) for the fix.

5. Database

5.1 Connection & Failover

config.py attempts a 3-second TCP connection to POSTGRES_URI on startup. If unreachable it falls back to POSTGRES_local_URI. The resolved URI is then assigned to SQLALCHEMY_DATABASE_URI. Note: there is a current bug where the resolved value is never returned from resolve_database_uri() — it must be patched before deployment (see Section 11).

Classification: Confidential

5.2 Schema & Tables

All tables live in the schema defined by the schema_name env var (default: vertex). Three tables are defined:

Table	Key Columns	Purpose
users	id (UUID PK), email, role, password_hash	One record per user; role controls home page card visibility
team	id (UUID PK), name (unique)	Master list of team names; seeded by create_tables.py
userteam	id (UUID PK), user_id (FK→users), email, team_name; UNIQUE(user_id, team_name)	Many-to-many mapping — a user can belong to multiple teams

5.3 Team Presets

The TEAM_PRESETS dictionary in models.py maps a preset name to a list of teams. This is used by the Excel seeder to assign teams to users in bulk via the preset column in seed_users.xlsx:

Preset	Teams Assigned
admin	All 13 teams including admin
scipher	l1_ops, scipher_ops, scipher_vx, scipher_fx, scipher_rx, soc, scipher_di, roc, scipher, ml-ops, apac, csm
roc	roc
apac	apac
roc-apac	roc, apac
csm	csm
l1_ops	l1_ops

5.4 Initialising the Database

Run create_tables.py from inside the v2/app/ directory to create the schema, tables, seed all team names, and bulk-create users from seed_users.xlsx:

```
cd v2/app
python create_tables.py

# Default admin user created from seed_users.xlsx:
#   Email:    admin@vertex.com
```


Classification: Confidential

```
#   Password: Admin@123   ← CHANGE IMMEDIATELY after first login
```

Note: The schema CREATE SCHEMA statement in create_tables.py is present in code comments but not actually executed. You must create the schema manually before running the script:

```
psql -U postgres -d postgres
CREATE SCHEMA IF NOT EXISTS vertex;   -- or your schema_name value
\q
```


Classification: Confidential

6. Application Architecture

6.1 App Factory

app/__init__.py exports create_app(). It loads .env, initialises SQLAlchemy, Flask-Login, registers three Blueprints (main, auth, errors), calls register_error_handlers() and register_security_features(), and injects app_name, app_version, and support_email as Jinja2 context globals on every page.

6.2 Blueprints

Blueprint	URL Prefix	Responsibility
main	(none)	Primary routes: home, team pages, public team, login, logout, admin redirect
auth	/admin	Admin-only routes: user CRUD, team assignment, admin panel
errors	(none)	HTTP error handlers (400–500), SQL injection detection, rotating file logger

6.3 URL Route Map

Route	Auth?	Description
GET /	Required	Home page — shows team cards filtered by user's role
GET /team/<team_name>	Required	Team page — shows links for that team (403 if user not in team)
GET /public/<team_name>	Required	Public view of a team — only rows with is_public=TRUE shown
GET /my-team	Required	Redirects to team page if single team, or select screen if multiple
GET /team/admin	Admin	Redirects to /admin/pane
GET+POST /login	Public	Login form and handler; redirects to / on success
GET /logout	Required	Clears session, redirects to /login
GET+POST /admin/pane	Admin	Admin dashboard: list all users and teams
GET+POST /admin/create_user	Admin	Create new user with role and team assignments
GET+POST /admin/edit_user/<id>	Admin	Edit user email, role, and team memberships
GET+POST /admin/delete_user/<id>	Admin	Delete user (cascades UserTeam records)

Classification: Confidential

7. Data Layer — Excel Workbook (team_links.xlsx)

All link content is driven entirely by the Excel workbook. No links are hardcoded in routes or templates (except some legacy cards in index.html and base.html — see Section 11). The workbook lives at two locations:

- v2/team_links.xlsx — canonical copy at repo root
- v2/app/static/data/team_links.xlsx — duplicate; this is the path referenced by xl_file in .env

7.1 Workbook Structure

Each sheet corresponds to one team. The sheet name is used as the team name if the Team column is empty. Confirmed sheets in the repo copy: index, l1_ops, soc, admin.

Column	Required?	Description
Team / Title	Yes	Display name shown on the card
URL	Yes	Full URL (https://...), domain only, or internal team path
Icon	No	Filename without extension — matched against static/data/logo/ (png/jpg/svg/webp)
Team	No	Overrides the sheet-name fallback for the Team column
category	No	Reserved for future filtering
is_public	No	TRUE/1/yes = visible on /public/<team> route without login
preview	No	Screenshot filename — matched in static/data/app-screen/
description	No	Tooltip / card description text

7.2 URL Resolution Logic (helpers.py)

The _process_url() function in helpers.py resolves each URL field:

- Starts with http:// or https:// → external link, opens in new tab
- Contains a dot and no spaces → treated as a bare domain, https:// prepended, new tab
- Anything else → treated as an internal team route name, passed to url_for('team_page')
- Empty → href set to #, no navigation

8. Roles & Team Access Control

8.1 Roles

Classification: Confidential

A user's role field controls which team cards are visible on the home page (index.html). Access to a team page is controlled separately by the UserTeam mapping — the route checks if team_name is in the user's assigned teams and returns 403 if not.

Role	Home Page Cards Shown
admin	All cards including admin panel
scipher	Full Scipher card set (13 teams)
roc	ROC card only
apac	APAC card only
roc-apac	ROC + APAC cards
csm	CSM card only
l1_ops	L1 Ops card only

8.2 All Available Teams

Defined in ALL_teams list in models.py: l1_ops, scipher_vx, scipher_fx, scipher_rx, scipher_ops, soc, scipher_di, roc, scipher, ml-ops, apac, csm, admin.

8.3 Public Team Routes

The /public/<team_name> route is accessible to any logged-in user regardless of their team membership. It only shows rows from the Excel sheet that have is_public set to a truthy value. Public sheets are: roc, apac, csm (defined in PUBLIC_SHEETS in app.py).

9. Security Features

9.1 Rate Limiting (security.py)

Flask-Limiter is initialised with in-memory storage. The following limits are applied:

Endpoint	Limit	Key
Global default	200 per day; 50 per hour	Client IP address
GET / (home)	30 per minute	Client IP address
GET /team/<name>	30 per minute	Client IP address
POST /login	5 per 15 minutes	Email address submitted (falls back to IP)

Classification: Confidential

⚠ Note: In-memory rate limit storage resets on every process restart. For persistent rate limiting across restarts or multiple workers, configure a Redis storage URI in security.py.

9.2 SQL Injection Detection (errors.py)

A before_request hook in the errors blueprint scans every incoming query string and form field for SQL keywords (UNION, SELECT, DROP, INSERT, UPDATE, DELETE, --, ;). Matches are logged as SQL-SUSPECT and return a 400 response. This is a basic heuristic guard — it does not replace parameterised queries.

9.3 Error Logging

register_error_handlers() sets up a RotatingFileHandler when the app is not in debug mode. Log file: {APP_NAME}-error_log.log in the working directory. Max size: 10 MB, 5 backups retained.

10. Running the Application

10.1 Development

```
cd v2
source venv/bin/activate
python run.py
# App runs at http://127.0.0.1:5000 in debug mode
```

10.2 Production (Gunicorn)

```
cd v2
source venv/bin/activate
gunicorn --workers 4 --bind 0.0.0.0:5000 wsgi:app

# Or with a config file:
gunicorn --config gunicorn_config.py wsgi:app
```

Note: run.py runs Flask in debug=True mode. Never use this in production. Always use wsgi.py with Gunicorn.

10.3 systemd Service (example)

```
[Unit]
Description=Vertex Team Links Portal
After=network.target

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/opt/vertex/v2
```


Classification: Confidential

```
EnvironmentFile=/opt/vertex/v2/.env
ExecStart=/opt/vertex/venv/bin/gunicorn --workers 4 --bind 0.0.0.0:5000 wsgi:app
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```


Classification: Confidential

11. Known Issues & Technical Debt

Issue	Severity	Detail & Fix
resolve_database_uri() never returns a value	Critical	config.py computes POSTGRES_URI locally but has no return statement — SQLALCHEMY_DATABASE_URI is always None. Fix: add 'return POSTGRES_URI' at the end of the function.
Schema not created by create_tables.py	High	The CREATE SCHEMA statement is commented out. The schema must be created manually in psql before running create_tables.py, otherwise table creation fails.
Hardcoded team links in base.html	Medium	base.html contains ~15 hardcoded url_for('main.team_page') links in the sidebar for specific teams. These should be driven from the DB or a config, not hard-wired in HTML.
index.html uses role string matching	Medium	Home page card visibility is controlled by Jinja2 'if role in [...]' checks — a second source of truth alongside the DB team mapping. Role changes in the DB do not automatically update visible cards without also updating index.html.
Duplicate team_links.xlsx	Low	The workbook exists at both v2/team_links.xlsx and v2/app/static/data/team_links.xlsx. Only the latter is loaded at runtime (via xl_file env var). They can easily drift out of sync.
helpers copy.py stale file	Low	'helpers copy.py' is an old version of helpers.py — safe to delete from the repo.
In-memory rate limiting	Low	Flask-Limiter uses memory:// storage — rate limit counters reset on restart and are not shared across Gunicorn workers. For multi-worker production: set storage_uri to a Redis instance.
run.py uses debug=True	High	The development entry point runs Flask with debug=True — this enables the interactive debugger and must NEVER be used in production. Always deploy via wsgi.py + Gunicorn.
No password reset flow	Medium	There is no self-service or admin-initiated password reset UI. Password changes require direct DB access or re-seeding.
select_team.html template missing	Medium	The /my-team route renders 'select_team.html' when a user has multiple teams, but this template does not exist in the templates/ directory — will cause a 500 error.

12. Immediate Actions for Incoming Developer

Classification: Confidential

- 🔴 **CRITICAL: Fix resolve_database_uri() in config.py — add 'return POSTGRES_URI' before the class definition**
- 🔴 **CRITICAL: Create the PostgreSQL schema manually before running create_tables.py**
- 🔴 **CRITICAL: Change the default admin password (admin@vertex.com / Admin@123)**
- 🔴 **CRITICAL: Change session_secret_key in .env to a strong random string**
- ⚠ Create select_team.html template — the /my-team route will 500 without it
- ⚠ Remove hardcoded team links from base.html sidebar — drive from DB
- ⚠ Unify team_links.xlsx — keep one copy and update xl_file env var accordingly
- ⚠ Never deploy using run.py — use wsgi.py + Gunicorn in production
- 🧹 Delete helpers copy.py from the repo
- 🧹 Investigate and remove the E9A3E500 binary file
- 📋 Consider Redis-backed rate limiting for multi-worker production deployments
- 📋 Add a password reset flow (admin-initiated at minimum)

Handover Sign-Off

By signing below, both parties confirm that the handover of the Vertex Team Links Portal project has been completed and that the incoming developer has received all necessary access, documentation, and context.

Outgoing Developer	**Incoming Developer**
Name: _____	Name: _____
Date: _____	Date: _____

Classification: Confidential